

oops

December 11, 2025

- 0.1 Write a class called Product. The class should have fields called name, amount, and price, holding the product's name, the number of items of that product in stock, and the regular price of the product. There should be a method get_price that receives the number of items to be bought and returns the cost of buying that many items, where the regular price is charged for orders of less than 10 items, a 10% discount is applied for orders of between 10 and 99 items, and a 20% discount is applied for orders of 100 or more items. There should also be a method called make_purchase that receives the number of items to be bought and decreases amount by that much.

```
[1]: class Product:

    def __init__(self, name, amount, price):
        self.name = name
        self.amount = amount
        self.price = price

    def get_price(self, number_to_be_bought):
        discount = 0
        if number_to_be_bought < 10:
            pass
        elif 10 <= number_to_be_bought < 99:
            discount = 10
        else:
            discount = 20
        price = (100 - discount) / 100 * self.price
        return price * number_to_be_bought

    def make_purchase(self, quantity):
        self.amount -= quantity

# name = input('name:')
# amount = int(input('Digit amount of items'))
# price = int(input('Digit price of items'))

name, amount, price = 'shoes', 200, 33
```

```

shoes = Product(name, amount, price)
# quantity = int(input('Digit amount of items to buy'))

q1 = 4
print(f'cost for {q1} {shoes.name} = {shoes.get_price(q1)}')
shoes.make_purchase(q1)
print(f'remaining stock: {shoes.amount}\n')

q2 = 12
print(f'cost for {q2} {shoes.name} = {shoes.get_price(q2)}')
shoes.make_purchase(q2)
print(f'remaining stock: {shoes.amount}\n')

q3 = 112
print(f'cost for {q3} {shoes.name} = {shoes.get_price(q3)}')
shoes.make_purchase(q3)
print(f'remaining stock: {shoes.amount}\n')

```

cost for 4 shoes = 132.0
remaining stock: 196

cost for 12 shoes = 356.4
remaining stock: 184

cost for 112 shoes = 2956.8
remaining stock: 72

0.2 You need to create the foundations of an e-commerce engine for a B2C (business-to-consumer) retailer. You need to have a class for a customer called User, a class for items in inventory called Item, and a shopping cart class called Cart. Items go in Carts, and Users can have multiple Carts. Also, multiple items can go into Carts, including more than one of any single item.

```

[2]: class User:
    def __init__(self, id, name):
        self.id = id
        self.name = name
    def display_user(self):
        print('ID', self.id, 'Name:', self.name)

class Item:
    def __init__(self, id, name, price, sold, available):
        self.id = id
        self.name = name

```

```

        self.price = price
        self.sold = sold
        self.available = available

class Cart:
    def __init__(self, user):
        self.user = user
        self.cart_items = []
    def insert_items(self, item, quantity):
        for i in range(quantity):
            if item.available == 0:
                print('Out of stock')
                break
            self.cart_items.append(item)
            item.sold += 1
            item.available -= 1
    def display_cart(self):
        print('This Cart belongs to', self.user.name, 'with ID', self.user.id)
        self.total = 0
        for i in self.cart_items:
            print('Item', i.name)
            self.total += i.price
        print('Total price = ', self.total)

#Creating a user
user1 = User(1, 'Kavit')
user1.display_user()

#Creating items
apple = Item(1, 'apple', 100, 0, 10)
greencoconut = Item(2, 'green coconut', 150, 0, 3)
milk = Item(2, 'milk', 24, 0, 100)

#Creating two carts for user1
cart1 = Cart(user1)
cart2 = Cart(user1)

#Adding items to cart1
cart1.insert_items(apple, 2)
cart1.insert_items(milk, 3)

#Adding items to cart2
cart2.insert_items(greencoconut, 3)
cart2.insert_items(milk, 20)

```

Vishal Acharya

Vishal Acharya

Vishal Acharya

```

        self.cost += 50
    elif self.size == 'medium':
        self.cost += 100
    else:
        self.cost += 200
    topping_prices_20 = ['corn', 'tomato', 'onion', 'capsicum']
    topping_prices_50 = ['mushroom', 'olives', 'broccoli']
    for topping in self.toppings:
        if topping in topping_prices_20:
            self.cost += 20
        else:
            self.cost += 50
    #for cheese
    self.cost += 50 * len(self.cheese)
    return self.cost

class Order:
    def __init__(self, name, customerid):
        self.name = name
        self.customerid = customerid

    def order(self, n):
        self.pizzas = []
        for i in range(n):
            toppings = []
            cheese = []
            print('Customize Pizza', i+1)
            size = input('Select size: ')
            t = int(input('How many toppings: '))
            for i in range(t):
                toppings.append(input('Enter toppings: '))
            t = int(input('How many cheese: '))
            for i in range(t):
                cheese.append(input('Enter cheese: '))
            self.pizzas.append(Pizza(size, toppings, cheese))

    def bill(self):
        self.total = 0
        count = 1
        for p in self.pizzas:
            print('Pizza', count)
            print(p.size, p.toppings, p.cheese)
            self.total += p.price()
            count += 1
        print('Total bill amount:', self.total)

number=int(input("How many pizzas you want to order: "))

```

```
order1 = Order('Kavit', 1)
order1.order(number)
order1.bill()
```

How many pizzas you want to order: 2
Customize Pizza 1
Select size: small
How many toppings: 1
Enter toppings: corn
How many cheese: 0
Customize Pizza 2
Select size: small
How many toppings: 0
How many cheese: 0
Pizza 1
small ['corn'] []
Pizza 2
small [] []
Total bill amount: 120

0.3 Write a program to build a simple Student Management System using Python which can perform the following operations:

1. Accept
2. Display
3. Search
4. Delete
5. Update

0.3.1 Accept – This method takes details from the user like name, roll number, and marks for two different subjects.

0.3.2 Display – This method displays the details of every student.

0.3.3 Search – This method searches for a particular student from the list of students. This method will ask the user for roll number and then search according to the roll number

0.3.4 Delete – This method deletes the record of a particular student with a matching roll number.

0.3.5 Update – This method updates the roll number of the student. This method will ask for the old roll number and new roll number. It will replace the old roll number with a new roll number.

```
[ ]: # This is simplest Student data management program in python

# Create class "Student"
class Student:
```

```

# Constructor
def __init__(self, name, rollno, m1, m2):
    self.name = name
    self.rollno = rollno
    self.m1 = m1
    self.m2 = m2

# Function to create and append new student
def accept(self):
    Name=input("Enter Name: ")
    Rollno=int(input("Enter Rollno.: "))
    marks1=int(input("Enter marks of 1: "))
    marks2=int(input("Enter marks of 2: "))
    ob = Student(Name, Rollno, marks1, marks2)
    ls.append(ob)

# Function to display student details
def display(self, ob):
    print("Name : ", ob.name)
    print("RollNo : ", ob.rollno)
    print("Marks1 : ", ob.m1)
    print("Marks2 : ", ob.m2)
    print("\n")

# Search Function
def search(self, rn):
    for i in range(ls.__len__()):
        if(ls[i].rollno == rn):
            return i

# Delete Function
def delete(self, rn):
    i = obj.search(rn)
    del ls[i]

# Update Function
def update(self, rn, No):
    i = obj.search(rn)
    roll = No
    ls[i].rollno = roll

# Create a list to add Students
ls = []
# an object of Student class
obj = Student('', 0, 0, 0)

```

```
print("\nOperations used, ")
print("\n1.Accept Student details\n2.Display Student Details\n3.Search Details of a Student\n4.Delete Details of Student\n5.Update Student Details\n6.Exit")

# ch = int(input("Enter choice:"))
# if(ch == 1):
obj.accept()
obj.accept()
obj.accept()

# elif(ch == 2):
print("\n")
print("\nList of Students\n")
for i in range(ls.__len__()):
    obj.display(ls[i])

# elif(ch == 3):
print("\n Student Found, ")
s = obj.search(10)
obj.display(ls[s])

# elif(ch == 4):
obj.delete(30)
print(ls.__len__())
print("List after deletion")
for i in range(ls.__len__()):
    obj.display(ls[i])

# elif(ch == 5):
obj.update(20,15)
print(ls.__len__())
print("List after updation")
for i in range(ls.__len__()):
    obj.display(ls[i])

# else:
print("Thank You !")
```

0.4 Stacks and Queues. Write a class which defines a data structure that can behave as both a queue (FIFO) or a stack (LIFO), There are four methods that should be implemented:

0.4.1 shift() returns the first element and removes it from the list

0.4.2 unshift() “pushes” a new element to the front or head of the list

0.4.3 push() adds a new element to the end of a list

0.4.4 pop() returns the last element and removes it from the list

```
[ ]: class StackQueue:
    def __init__(self, L):
        self.L = L
    def shift(self):
        if len(self.L) == 0:
            pass
            raise Exception('List is empty')
        try:
            x = self.L.pop(0)
            return x
        except Exception:
            print(Exception)
    def unshift(self, n):
        self.L.insert(0, n)
    def push(self, n):
        self.L.append(n)
    def pop(self):
        if len(self.L) == 0:
            raise Exception('List is empty')
        try:
            x = self.L.pop()
            return x
        except Exception:
            print(Exception)
    def display(self):
        return self.L

sq = StackQueue([1,4,6,8,9])
print(sq.shift())
sq.unshift(7)
sq.push(10)
print(sq.pop())
print(sq.display())
```

1 Write the definition of a Point class. Objects from this class should have a

a method show to display the coordinates of the point

a method move to change these coordinates.

a method dist that computes the distance between 2 points.

```
[ ]: import math

class Point(object):
    """Class to handle point in a 2 dimensions space"""

    def __init__(self, x, y):
        """
        :param x: the value on the X-axis
        :type x: float
        :param y: the value on the Y-axis
        :type y: float
        """
        self.x = x
        self.y = y

    def show(self):
        """
        :return: the coordinate of this point
        :rtype: a tuple of 2 elements (float, float)
        """
        return self.x, self.y

    def move(self, x, y):
        """
        :param x: the value to move on the X-axis
        :type x: float
        :param y: the value to move on the Y-axis
        :type y: float
        """
        self.x += x
        self.y += y

    def dist(self, pt):
        """
        :param pt: the point to compute the distance with
```

```

        :type pt: :class:`Point` object
        :return: the distance between this point ant pt
        :rtype: int
        """
        dx = pt.x - self.x
        dy = pt.y - self.y
        return math.sqrt(dx ** 2 + dy ** 2)

```

```

[ ]: p1 = Point(2, 3)
      p2 = Point(3, 3)
      print(p1.show())
      print(p2.show())
      p1.move(10, -10)
      print(p1.show())
      print(p2.show())
      print(p1.dist(p2))

```

2 Get index in the list of objects by attribute in Python

```

[ ]: class X:
      def __init__(self, val):
          self.val = val

      def getIndex(li, target):
          for index, x in enumerate(li):
              if x.val == target:
                  return index
          return -1

      # Driver code
      li = [1,2,3,4,5,6]

      # Converting all the items in
      # list to object of class X
      a = list()
      for i in li:
          a.append(X(i))

      print(getIndex(a,3))

```

3 How to create a list of object in Python class

```

[ ]: class geeks:
      def __init__(self, name, roll):
          self.name = name

```

```

        self.roll = roll

# creating list
list = []

# appending instances to list
list.append(geeks('Akash', 2))
list.append(geeks('Deependra', 40))
list.append(geeks('Reaper', 44))
list.append(geeks('veer', 67))

# Accessing object value using a for loop
for obj in list:
    print(obj.name, obj.roll, sep=' ')

print("")
# Accessing individual elements
print(list[0].name)
print(list[1].name)
print(list[2].name)
print(list[3].name)

```

4 Create a Sphere class that accepts a radius upon instantiation and has a volume and surface area method.

```

[ ]: class Sphere():

    def __init__(self, radius):
        self.radius = radius

    def volume(self):
        return (4/3)*3.14*self.radius**3

    def surface_area(self):
        return 4 * 3.14 * self.radius **2

```

```

[ ]: s = Sphere(1)
print(s.surface_area())
print(s.volume())

```

Write a Rectangle class in Python language, allowing you to build a rectangle with length and width attributes.

Create a Perimeter() method to calculate the perimeter of the rectangle and a Area() method to calculate the area of the rectangle.

Create a method display() that display the length, width, perimeter and area of an object created using an instantiation on rectangle class.

```
[ ]: class Rectangle:
    # define constructor with attributes: length and width
    def __init__(self, length , width):
        self.length = length
        self.width = width

    # Create Perimeter method
    def Perimeter(self):
        return 2*(self.length + self.width)

    # Create area method
    def Area(self):
        return self.length*self.width

    # create display method
    def display(self):
        print("The length of rectangle is: ", self.length)
        print("The width of rectangle is: ", self.width)
        print("The perimeter of rectangle is: ", self.Perimeter())
        print("The area of rectangle is: ", self.Area())
```

```
[ ]: myRectangle = Rectangle(7 , 5)
myRectangle.display()
print("-----")
```

5 Create a Python class called BankAccount which represents a bank account, having as attributes: accountNumber (numeric type), name (name of the account owner as string type), balance.

Create a constructor with parameters: accountNumber, name, balance.

Create a Deposit() method which manages the deposit actions.

Create a Withdrawal() method which manages withdrawals actions.

Create an bankFees() method to apply the bank fees with a percentage of 5% of the balance account.

Create a display() method to display account details.

Give the complete code for the BankAccount class.

```
[ ]: class BankAccount:
    # create the constructor with parameters: accountNumber, name and balance
    def __init__(self,accountNumber, name, balance):
        self.accountNumber = accountNumber
        self.name = name
        self.balance = balance
```

```

# create Deposit() method
def Deposit(self , d ):
    self.balance = self.balance + d

# create Withdrawal method
def Withdrawal(self , w):
    if(self.balance < w):
        print("impossible operation! Insufficient balance !")
    else:
        self.balance = self.balance - w

# create bankFees() method
def bankFees(self):
    self.balance = (95/100)*self.balance

# create display() method
def display(self):
    print("Account Number : " , self.accountNumber)
    print("Account Name : " , self.name)
    print("Account Balance : " , self.balance , " $")

# Testing the code :
newAccount = BankAccount(2178514584, "Albert" , 2700)
# Creating Withdrawal Test
newAccount.Withdrawal(300)
# Create deposit test
newAccount.Deposit(200)
# Display account informations
newAccount.display()

```

6 1 - Create a Coputation class with a default constructor (without parameters) allowing to perform various calculations on integers numbers.

2 - Create a method called Factorial() which allows to calculate the factorial of an integer. Test the method by instantiating the class.

3 - Create a method called Sum() allowing to calculate the sum of the first n integers $1 + 2 + 3 + \dots + n$. Test this method.

4 - Create a method called testPrim() in the Calculation class to test the primality of a given integer. Test this method.

4 - Create a method called testPrims() allowing to test if two numbers are prime between them.

5 - Create a tableMult() method which creates and displays the multiplication table of a given integer. Then create an allTablesMult() method to display all the integer multiplication tables 1, 2, 3, ..., 9.

6 - Create a static listDiv() method that gets all the divisors of a given integer on new list called Ldiv. Create another listDivPrim() method that gets all the prime divisors of a given integer.

```
[ ]: class Computation:
    def __init__(self):
        pass
    # --- Factorial -----
    def factorial(self, n):
        j = 1
        for i in range (1, n + 1):
            j = j * i
        return j

    # --- Sum of the first n numbers ----
    def sum (self, n):
        j = 1
        for i in range (1, n + 1):
            j = j + i
        return j

    # --- Primality test of a number -----
    def testPrim (self, n):
        j = 0
        for i in range (1, n + 1):
            if (n% i == 0):
                j = j + 1
        if (j == 2):
            return True
        else:
            return False

    # --- Primality test of two integers -----
    def testprims (self, n, m):

        # initialize the number of commons divisors
        commonDiv = 0
        for i in range (1, n + 1):
            if (n% i == 0 and m% i == 0):
                commonDiv = commonDiv + 1
        if commonDiv == 1:
            print ("The numbers", n, "and", m, "are co-primes")
        else:
            print ("The numbers", n, "and", m, "are not co-primes")

    #---Multiplication table-----
    def tableMult (self, k):
        for i in range (1,10):
```

```

        print (i, "x", k, "=", i * k)

# --- All multiplication tables of the numbers 1, 2, ..., 9
def allTables (self):
    for k in range (1,10):
        print ("\nthe multiplication table of:", k, "is:")
        for i in range (1,10):
            print (i, "x", k, "=", i * k)

# ----- list of divisors of an integer
def listDiv (self, n):
    # initialization of the list of divisors
    lDiv = []
    for i in range (1, n + 1):
        if (n% i == 0):
            lDiv.append (i)
    return lDiv

# ----- list of prime divisors of an integer -----
def listDivPrim (self, n):
    # initialization of the list of divisors
    lDiv = []
    for i in range (1, n + 1):
        if (n% i == 0 and self.testPrim (i)):
            lDiv.append (i)
    return lDiv

# Instantiation example
Comput= Computation ()
Comput.testprims (13, 7)
print ("List of divisors of 18:", Comput.listDiv (18))
print ("List of prime divisors of 18:", Comput.listDivPrim (18))
Comput.allTables ()

```

```

[1]: #write a python program that has class store which keeps record of code and
    ↳ price of
#each product. Display a menu of all products to the user and prompt
#him to enter the quantity of each item required . generate a bill and display
    ↳ total amount.
class Store:
    def __init__(self,n):
        self.item_code=[]
        self.price=[]
        self.n=n
    def get_data(self):
        #n=int(input("enter no of items: "))
        for i in range(self.n):

```

```

        self.item_code.append(int(input("enter code of item: ")))
        self.price.append(int(input("enter cost of item: ")))
    def display_data(self):
        print("Item Code \t Price")
        for i in range(self.n):
            print(self.item_code[i], "\t\t", self.price[i])

    def calculate_bill(self, quant):
        total_amount=0
        for i in range(self.n):
            total_amount+=(self.price[i]*quant[i])
        print("*****Bill*****")
        print("ITEM \t PRICE \t QUANTITY \t SUBTOTAL")
        for i in range(self.n):
            print(self.item_code[i], "\t", self.price[i], "\t", quant[i], "\t\t",
                  self.price[i]*quant[i])
        print("*****")
        print("Total= ", total_amount)

n=int(input("enter no of items: "))
s1=Store(n)
s1.get_data()
s1.display_data()
q=[]
print("Enter quantity of each item: ")
for i in range(n):
    q.append(int(input("Enter quantity of item {} : ".format(i+1))))

s1.calculate_bill(q)

```

```

enter no of items: 1
enter code of item: 1
enter cost of item: 25
Item Code      Price
1              25
Enter quantity of each item:
Enter quantity of item 1 : 5
*****Bill*****
ITEM    PRICE    QUANTITY    SUBTOTAL
1       25       5           125
*****
Total=  125

```

[2]: *#write a python program for library book record with oops.*

```
class library:
```

```

def __init__(self):
    self.title=""
    self.author=""
    self.publisher=""
def read(self):
    self.title=input("Enter Book Title: ")
    self.author=input("Enter Book author: ")
    self.publisher=input("Enter Book Publisher: ")
def display(self):
    print("Title:", self.title)
    print("Author:", self.author)
    print("Publisher:", self.publisher)
    print("\n")
my_book=[]
ch='y'
while(ch=='y'):
    print('
1. Add New Book
2. Display Books
')
    choice=int(input("Enter choice: "))
    if(choice==1):
        book=library()
        book.read()
        my_book.append(book)
    elif(choice==2):
        for i in my_book:
            i.display()
    else:
        print("Invalid choice!")
    ch=input("Do you want to continue..?")
print("Bye!")

```

1. Add New Book
2. Display Books

```

Enter choice: 1
Enter Book Title: vha
Enter Book author: vishal
Enter Book Publisher: lj
Do you want to continue..?2
Bye!

```

7 Create a single Python class named EnrollmentSystem that manages a mini university enrollment system.

You must not use multiple classes — all logic must be inside this one class.

The system must support:

Class Variables

A global transaction log for all actions

A counter that increases each time the system object is created

Constructor

Initialize storage for all courses and students

Store term name

Log creation event

Store Data Using Dictionaries (Inside the Same Class)

Course data: code, title, credits, capacity, schedule, enrolled list, waitlist

Student data: id, name, max credits, enrolled courses

Methods to Implement

Register Courses

Register Students

Enroll Student in a Course with rules:

Check credit limit

Check schedule conflicts

Add to enrolled list if seat available

Otherwise add to waitlist

Drop Student from a Course

Remove from enrolled or waitlist

Promote first student from waitlist if possible

Check Time Conflicts

Calculate Student's Current Credits

Generate Reports

Student report

Course report

System audit

Features to Test

Enrollment with conflict checking
Credit limit enforcement
Waitlist FIFO and automatic promotion
Detailed logging for every action

```
[1]: """
EnrollmentSystem_single_class.py
Single-class implementation of a mini university enrollment manager.

Features (all inside the EnrollmentSystem class):
- Class variables: transaction_log, system_count
- Register courses & students (stored as dicts)
- Enroll/drop with credit checks, schedule-conflict detection
- Course waitlist (FIFO) and automatic promotion on drop
- Detailed logging with timestamps

No private attributes, no helper classes, no @staticmethod/@classmethod, no_
↳ inheritance.

Sample usage and expected output are provided at the bottom inside the
`if __name__ == "__main__":` block and in the final comment block.
"""

from datetime import datetime

class EnrollmentSystem:
    # Class variables
    transaction_log = [] # global log of string messages
    system_count = 0 # increments each time an EnrollmentSystem is_
    ↳ instantiated

    def __init__(self, term_name="Term1"):
        """Initialize registries and increment system counter."""
        EnrollmentSystem.system_count += 1
        self.term = term_name
        # course_code -> course_dict
        self.courses = {}
        # student_id -> student_dict
        self.students = {}
        # instance-level seq number for nicer logs
        self._local_txn_counter = 0
        self._log(f"Created EnrollmentSystem for term '{self.term}'")

    # ----- Utilities / Helpers -----
```

```

def _now_str(self):
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

def _log(self, message):
    """Append a timestamped entry to class transaction_log."""
    self._local_txn_counter += 1
    entry = f"[{EnrollmentSystem.system_count}].[self._local_txn_counter]]_
↪{self._now_str(): {message}"
    EnrollmentSystem.transaction_log.append(entry)

def _time_to_minutes(self, tstr):
    """Convert 'HH:MM' -> integer minutes since midnight."""
    h, m = tstr.split(":")
    return int(h) * 60 + int(m)

def is_conflict(self, schedule_a, schedule_b):
    """Return True if any timeslot in schedule_a conflicts with any in
↪schedule_b.

    schedule format: list of tuples (day, 'HH:MM', 'HH:MM')
    Touching endpoints are NOT conflicts: end == start is allowed.
    """
    for day_a, start_a, end_a in schedule_a:
        sa = self._time_to_minutes(start_a)
        ea = self._time_to_minutes(end_a)
        for day_b, start_b, end_b in schedule_b:
            if day_a != day_b:
                continue
            sb = self._time_to_minutes(start_b)
            eb = self._time_to_minutes(end_b)
            # overlap exists if start < other_end and other_start < end
            if sa < eb and sb < ea:
                return True
    return False

# ----- Registration -----
def register_course(self, code, title, credits, capacity, schedule):
    """Register a new course. Returns success message or error if duplicate.
↪"""
    code = code.strip().upper()
    if code in self.courses:
        msg = f"Course {code} already registered"
        self._log(msg)
        return msg
    # Normalize schedule to list of (DAY, HH:MM, HH:MM)
    normalized_schedule = []
    for day, start, end in schedule:

```

```

        normalized_schedule.append((day.strip().title(), start.strip(), end.
↪strip()))
        course = {
            'code': code,
            'title': title.strip(),
            'credits': int(credits),
            'capacity': int(capacity),
            'schedule': normalized_schedule,
            'enrolled': [],      # list of student_ids
            'waitlist': []       # list of student_ids (FIFO)
        }
        self.courses[code] = course
        msg = f"Registered course {code}: {title} (credits={credits},
↪cap={capacity})"
        self._log(msg)
        return msg

    def register_student(self, student_id, name, max_credits):
        """Register a new student. Returns success message or error if
↪duplicate."""
        sid = student_id.strip().upper()
        if sid in self.students:
            msg = f"Student {sid} already registered"
            self._log(msg)
            return msg
        student = {
            'id': sid,
            'name': name.strip(),
            'max_credits': int(max_credits),
            'enrolled_courses': [] # list of course codes
        }
        self.students[sid] = student
        msg = f"Registered student {sid}: {name} (max_credits={max_credits})"
        self._log(msg)
        return msg

    # ----- Credit bookkeeping -----
    def current_credits(self, student_id):
        """Sum credits of courses the student is currently enrolled in."""
        sid = student_id.strip().upper()
        if sid not in self.students:
            return 0
        total = 0
        for ccode in self.students[sid]['enrolled_courses']:
            if ccode in self.courses:
                total += self.courses[ccode]['credits']
        return total

```

```

# ----- Enrollment / Drop logic -----
def enroll(self, student_id, course_code):
    """Attempt to enroll student in course. Returns status string and logs
    the attempt."""
    sid = student_id.strip().upper()
    ccode = course_code.strip().upper()

    # Validate existence
    if sid not in self.students:
        msg = f"Enroll failed: Student {sid} not found"
        self._log(msg)
        return msg
    if ccode not in self.courses:
        msg = f"Enroll failed: Course {ccode} not found"
        self._log(msg)
        return msg

    student = self.students[sid]
    course = self.courses[ccode]

    # Already enrolled
    if ccode in student['enrolled_courses']:
        msg = f"already enrolled: {sid} -> {ccode}"
        self._log(msg)
        return msg
    # Already waitlisted
    if sid in course['waitlist']:
        msg = f"already waitlisted: {sid} -> {ccode}"
        self._log(msg)
        return msg

    # Credit check (only counts if would be enrolled immediately)
    current = self.current_credits(sid)
    if current + course['credits'] > student['max_credits']:
        msg = f"credit limit exceeded: {sid} (current {current}, trying
        to add {course['credits']} max {student['max_credits']}) -> {ccode}"
        self._log(msg)
        return "credit limit exceeded"

    # Schedule conflict check against all enrolled courses
    for enrolled_code in student['enrolled_courses']:
        other_course = self.courses.get(enrolled_code)
        if other_course and self.is_conflict(course['schedule'],
        other_course['schedule']):
            msg = f"schedule conflict: {sid} -> {ccode} conflicts with
            {enrolled_code}"

```

```

        self._log(msg)
        return "schedule conflict"

    # Seat available?
    if len(course['enrolled']) < course['capacity']:
        course['enrolled'].append(sid)
        student['enrolled_courses'].append(ccode)
        msg = f"enrolled: {sid} -> {ccode}"
        self._log(msg)
        return "enrolled"
    else:
        # Add to waitlist if not present
        course['waitlist'].append(sid)
        msg = f"waitlisted: {sid} -> {ccode}"
        self._log(msg)
        return "waitlisted"

    def drop(self, student_id, course_code):
        """Drop a student from a course. Handle waitlist promotion. Returns
        ↪ status string."""
        sid = student_id.strip().upper()
        ccode = course_code.strip().upper()

        if sid not in self.students:
            msg = f"Drop failed: Student {sid} not found"
            self._log(msg)
            return msg
        if ccode not in self.courses:
            msg = f"Drop failed: Course {ccode} not found"
            self._log(msg)
            return msg

        student = self.students[sid]
        course = self.courses[ccode]

        if sid in course['enrolled']:
            course['enrolled'].remove(sid)
            if ccode in student['enrolled_courses']:
                student['enrolled_courses'].remove(ccode)
            drop_msg = f"dropped: {sid} from {ccode}"
            self._log(drop_msg)
            # Attempt to promote from waitlist
            promoted = None
            while course['waitlist']:
                candidate_sid = course['waitlist'].pop(0) # FIFO
                candidate = self.students.get(candidate_sid)
                if candidate is None:

```

```

        # skip if candidate vanished
        continue
    # Check candidate's credit capacity before promoting
    cand_current = self.current_credits(candidate_sid)
    if cand_current + course['credits'] > candidate['max_credits']:
        # Can't promote due to credit limit; skip promotion for them
        skip_msg = f"promotion skipped for {candidate_sid} ->
↳{ccode} due to credit limit"
        self._log(skip_msg)
        continue
    # Also double-check schedule conflicts (rare but possible if
↳student enrolled elsewhere in meantime)
    conflict = False
    for other_code in candidate['enrolled_courses']:
        other_course = self.courses.get(other_code)
        if other_course and self.is_conflict(course['schedule'],
↳other_course['schedule']):
            conflict = True
            break
    if conflict:
        conflict_msg = f"promotion skipped for {candidate_sid} ->
↳{ccode} due to schedule conflict"
        self._log(conflict_msg)
        continue
    # Promote
    course['enrolled'].append(candidate_sid)
    candidate['enrolled_courses'].append(ccode)
    promoted = candidate_sid
    promo_msg = f"promoted: {candidate_sid} -> {ccode} from
↳waitlist"
    self._log(promo_msg)
    break
    if promoted:
        return f"dropped and promoted {promoted}"
    else:
        return "dropped"

elif sid in course['waitlist']:
    course['waitlist'].remove(sid)
    msg = f"removed from waitlist: {sid} -> {ccode}"
    self._log(msg)
    return "removed from waitlist"
else:
    msg = f"Drop failed: {sid} not enrolled or waitlisted in {ccode}"
    self._log(msg)
    return "not enrolled"

```

```

# ----- Reporting -----
def report_student(self, student_id):
    """Return a readable summary for the student."""
    sid = student_id.strip().upper()
    if sid not in self.students:
        return f"Student {sid} not found"
    s = self.students[sid]
    lines = []
    lines.append(f"Student {sid} - {s['name']}")
    lines.append(f"Max credits: {s['max_credits']}")
    lines.append(f"Currently enrolled credits: {self.current_credits(sid)}")
    lines.append("Enrolled courses:")
    if not s['enrolled_courses']:
        lines.append("  (none)")
    else:
        for code in s['enrolled_courses']:
            c = self.courses.get(code)
            if c:
                lines.append(f"  {code}: {c['title']} ({c['credits']}_
↳credits) - schedule: {c['schedule']}")
            else:
                lines.append(f"  {code}: (course not found)")
    report = "\n".join(lines)
    self._log(f"report_student: {sid}")
    return report

def report_course(self, course_code):
    """Return a readable summary for the course."""
    ccode = course_code.strip().upper()
    if ccode not in self.courses:
        return f"Course {ccode} not found"
    c = self.courses[ccode]
    lines = []
    lines.append(f"Course {ccode} - {c['title']}")
    lines.append(f"Credits: {c['credits']}, Capacity: {c['capacity']}")
    lines.append(f"Enrolled ({len(c['enrolled'])}): {c['enrolled']}")
    lines.append(f"Waitlist ({len(c['waitlist'])}): {c['waitlist']}")
    lines.append(f"Schedule: {c['schedule']}")
    report = "\n".join(lines)
    self._log(f"report_course: {ccode}")
    return report

def audit(self):
    """Return a short system summary."""
    num_courses = len(self.courses)
    num_students = len(self.students)
    num_txns = len(EnrollmentSystem.transaction_log)

```

```

        summary = f"Term: {self.term} | Courses: {num_courses} | Students: {num_students} | Transactions: {num_txns}"
        self._log("audit")
        return summary

# ----- Sample test script (when run as a program) -----
if __name__ == "__main__":
    sys = EnrollmentSystem(term_name="Autumn-2025")

    # Register courses
    print(sys.register_course("CS101", "Intro to Programming", credits=3, capacity=2,
                               schedule=[("Mon", "09:00", "10:30"), ("Wed", "09:00", "10:30")]))
    print(sys.register_course("MA101", "Calculus I", credits=4, capacity=2,
                               schedule=[("Mon", "10:30", "12:00"), ("Thu", "09:00", "10:30")]))
    print(sys.register_course("PH101", "Physics I", credits=3, capacity=1,
                               schedule=[("Mon", "09:30", "11:00")])) # overlaps CS101

    # Register students
    print(sys.register_student("S001", "Asha", max_credits=7))
    print(sys.register_student("S002", "Rohit", max_credits=6))
    print(sys.register_student("S003", "Priya", max_credits=9))

    # Enrollment attempts
    print("ENROLL RESULTS:")
    print(sys.enroll("S001", "CS101")) # expected: enrolled
    print(sys.enroll("S002", "CS101")) # expected: enrolled
    print(sys.enroll("S003", "CS101")) # expected: waitlisted (capacity 2)
    print(sys.enroll("S001", "PH101")) # expected: schedule conflict (overlap with CS101)
    print(sys.enroll("S001", "MA101")) # expected: enrolled (credits go from 3 -> 7)
    print(sys.enroll("S001", "PH101")) # expected: credit limit exceeded (would go > max)

    # Now drop one student and observe waitlist promotion
    print(sys.drop("S002", "CS101")) # expected: S002 removed, S003 auto-enrolled from waitlist

    # Reports
    print("\nCOURSE REPORT CS101:\n", sys.report_course("CS101"))

```

```

print("\nSTUDENT REPORT S003:\n", sys.report_student("S003"))
print("\nAUDIT:\n", sys.audit())

# Finally, print the transaction log
print("\nTRANSACTION LOG:")
for entry in EnrollmentSystem.transaction_log:
    print(entry)

# ----- Sample expected output (illustrative)
↳ -----
"""
(Note: timestamps will vary.)

Registered course CS101: Intro to Programming (credits=3, cap=2)
Registered course MA101: Calculus I (credits=4, cap=2)
Registered course PH101: Physics I (credits=3, cap=1)
Registered student S001: Asha (max_credits=7)
Registered student S002: Rohit (max_credits=6)
Registered student S003: Priya (max_credits=9)
ENROLL RESULTS:
enrolled
enrolled
waitlisted
schedule conflict
enrolled
credit limit exceeded
dropped and promoted S003

COURSE REPORT CS101:
  Course CS101 - Intro to Programming
  Credits: 3, Capacity: 2
  Enrolled (2): ['S001', 'S003']
  Waitlist (0): []
  Schedule: [('Mon', '09:00', '10:30'), ('Wed', '09:00', '10:30')]

STUDENT REPORT S003:
  Student S003 - Priya
  Max credits: 9
  Currently enrolled credits: 3
  Enrolled courses:
    CS101: Intro to Programming (3 credits) - schedule: [('Mon', '09:00', '10:
↳ 30'), ('Wed', '09:00', '10:30')]

AUDIT:
  Term: Autumn-2025 | Courses: 3 | Students: 3 | Transactions: 13

```

TRANSACTION LOG:

*[1.1] 2025-12-11 10:xx: Created EnrollmentSystem for term 'Autumn-2025'
... (more log lines with timestamps for each operation) ...
"""*

Registered course CS101: Intro to Programming (credits=3, cap=2)

Registered course MA101: Calculus I (credits=4, cap=2)

Registered course PH101: Physics I (credits=3, cap=1)

Registered student S001: Asha (max_credits=7)

Registered student S002: Rohit (max_credits=6)

Registered student S003: Priya (max_credits=9)

ENROLL RESULTS:

enrolled

enrolled

waitlisted

schedule conflict

enrolled

credit limit exceeded

dropped and promoted S003

COURSE REPORT CS101:

Course CS101 - Intro to Programming

Credits: 3, Capacity: 2

Enrolled (2): ['S001', 'S003']

Waitlist (0): []

Schedule: [('Mon', '09:00', '10:30'), ('Wed', '09:00', '10:30')]

STUDENT REPORT S003:

Student S003 - Priya

Max credits: 9

Currently enrolled credits: 3

Enrolled courses:

CS101: Intro to Programming (3 credits) - schedule: [('Mon', '09:00', '10:30'), ('Wed', '09:00', '10:30')]

AUDIT:

Term: Autumn-2025 | Courses: 3 | Students: 3 | Transactions: 17

TRANSACTION LOG:

[1.1] 2025-12-11 10:08:53: Created EnrollmentSystem for term 'Autumn-2025'

[1.2] 2025-12-11 10:08:53: Registered course CS101: Intro to Programming (credits=3, cap=2)

[1.3] 2025-12-11 10:08:53: Registered course MA101: Calculus I (credits=4, cap=2)

[1.4] 2025-12-11 10:08:53: Registered course PH101: Physics I (credits=3, cap=1)

[1.5] 2025-12-11 10:08:53: Registered student S001: Asha (max_credits=7)

[1.6] 2025-12-11 10:08:53: Registered student S002: Rohit (max_credits=6)

[1.7] 2025-12-11 10:08:53: Registered student S003: Priya (max_credits=9)

```

[1.8] 2025-12-11 10:08:53: enrolled: S001 -> CS101
[1.9] 2025-12-11 10:08:53: enrolled: S002 -> CS101
[1.10] 2025-12-11 10:08:53: waitlisted: S003 -> CS101
[1.11] 2025-12-11 10:08:53: schedule conflict: S001 -> PH101 conflicts with
CS101
[1.12] 2025-12-11 10:08:53: enrolled: S001 -> MA101
[1.13] 2025-12-11 10:08:53: credit limit exceeded: S001 (current 7, trying +3
max 7) -> PH101
[1.14] 2025-12-11 10:08:53: dropped: S002 from CS101
[1.15] 2025-12-11 10:08:53: promoted: S003 -> CS101 from waitlist
[1.16] 2025-12-11 10:08:53: report_course: CS101
[1.17] 2025-12-11 10:08:53: report_student: S003
[1.18] 2025-12-11 10:08:53: audit

```

```

[1]: "\n(Note: timestamps will vary.)\n\nRegistered course CS101: Intro to
Programming (credits=3, cap=2)\nRegistered course MA101: Calculus I (credits=4,
cap=2)\nRegistered course PH101: Physics I (credits=3, cap=1)\nRegistered
student S001: Asha (max_credits=7)\nRegistered student S002: Rohit
(max_credits=6)\nRegistered student S003: Priya (max_credits=9)\nENROLL
RESULTS:\nenrolled\nenrolled\nwaitlisted\nschedule conflict\nenrolled\ncredit
limit exceeded\ndropped and promoted S003\n\nCOURSE REPORT CS101:\n Course CS101
- Intro to Programming\n Credits: 3, Capacity: 2\n Enrolled (2): ['S001',
'S003']\n Waitlist (0): []\n Schedule: [('Mon', '09:00', '10:30'), ('Wed',
'09:00', '10:30')]\n\nSTUDENT REPORT S003:\n Student S003 - Priya\n Max credits:
9\n Currently enrolled credits: 3\n Enrolled courses:\n   CS101: Intro to
Programming (3 credits) - schedule: [('Mon', '09:00', '10:30'), ('Wed', '09:00',
'10:30')]\n\nAUDIT:\n Term: Autumn-2025 | Courses: 3 | Students: 3 |
Transactions: 13\n\nTRANSACTION LOG:\n[1.1] 2025-12-11 10:xx: Created
EnrollmentSystem for term 'Autumn-2025'\n... (more log lines with timestamps for
each operation) ...\n"

```

```

[2]: %%writefile score.txt
India 397 England 280
Pakistan 277 USA 217
Australia 243 Afghanistan 225
SouthAfrica 352 SriLanka 232
NewZealand 250 Bangladesh 248
Netherlands 200 Pakistan 198
India 336 Bangladesh 188
England 280 Bangladesh 230
Australia 367 Afghanistan 123
SouthAfrica 212 NewZealand 210
Pakistan 190 India 191
SriLanka 150 Netherlands 151
Australia 289 England 290
USA 220 Netherlands 219
Bangladesh 175 SriLanka 176

```

```

India 229 Australia 230
England 198 SouthAfrica 197
NewZealand 301 Pakistan 299
Afghanistan 164 USA 165
Netherlands 310 India 312
SriLanka 250 Pakistan 251
Bangladesh 260 Netherlands 261
SouthAfrica 330 Australia 331
England 340 NewZealand 338

```

Writing score.txt

```

[4]: class WorldCupAnalyzer:
    # class variables
    instance_count = 0
    logs = []

    def __init__(self, filename):
        WorldCupAnalyzer.instance_count += 1
        self.filename = filename
        self.matches = []
        self.teams = []
        self.counter = 0
        self.loadfile()
        self.log("Analyzer ready with " + str(len(self.matches)) + " matches")

    def log(self, message):
        self.counter += 1
        entry = "[" + str(WorldCupAnalyzer.instance_count) + "." + str(self.
↵counter) + "]" + message
        WorldCupAnalyzer.logs.append(entry)

    def parse_line(self, line):
        parts = line.split()
        if len(parts) < 4:
            return None
        t1 = parts[0]
        s1 = int(parts[1])
        t2 = parts[2]
        s2 = int(parts[3])
        match = {}
        match["team1"] = t1
        match["score1"] = s1
        match["team2"] = t2
        match["score2"] = s2
        match["raw"] = line

```

```

    if s1 > s2:
        match["winner"] = t1
        match["margin"] = s1 - s2
    elif s2 > s1:
        match["winner"] = t2
        match["margin"] = s2 - s1
    else:
        match["winner"] = None
        match["margin"] = 0
    return match

def loadfile(self):
    f = open(self.filename, "r")
    for line in f:
        line = line.strip()
        if line == "":
            continue
        m = self.parse_line(line)
        if m is not None:
            self.matches.append(m)
            if m["team1"] not in self.teams:
                self.teams.append(m["team1"])
            if m["team2"] not in self.teams:
                self.teams.append(m["team2"])
    f.close()

# ----- ANALYSIS TASKS -----

def total_matches(self):
    return len(self.matches)

def list_teams(self):
    return self.teams

def matches_played(self):
    d = {}
    for t in self.teams:
        d[t] = 0
    for m in self.matches:
        d[m["team1"]] += 1
        d[m["team2"]] += 1
    return d

def total_runs(self):
    d = {}
    for t in self.teams:
        d[t] = 0

```

```

    for m in self.matches:
        d[m["team1"]] += m["score1"]
        d[m["team2"]] += m["score2"]
    return d

def wins_losses(self):
    d = {}
    for t in self.teams:
        d[t] = {"wins":0, "losses":0, "ties":0}
    for m in self.matches:
        if m["winner"] is None:
            d[m["team1"]]["ties"] += 1
            d[m["team2"]]["ties"] += 1
        else:
            d[m["winner"]]["wins"] += 1
            if m["winner"] == m["team1"]:
                d[m["team2"]]["losses"] += 1
            else:
                d[m["team1"]]["losses"] += 1
    return d

def highest_score(self):
    best = None
    best_score = -1
    for m in self.matches:
        if m["score1"] > best_score:
            best_score = m["score1"]
            best = m
        if m["score2"] > best_score:
            best_score = m["score2"]
            best = m
    return best

def highest_total(self):
    best = None
    best_total = -1
    for m in self.matches:
        total = m["score1"] + m["score2"]
        if total > best_total:
            best_total = total
            best = m
    return best

def largest_margin(self):
    best = None
    best_margin = -1
    for m in self.matches:

```

```

        if m["margin"] > best_margin:
            best_margin = m["margin"]
            best = m
    return best

# ----- REPORT -----

def report(self):
    print("Total matches:", str(self.total_matches()))
    print("Teams:", str(self.list_teams()))
    print("Matches played:", str(self.matches_played()))
    print("Total runs:", str(self.total_runs()))
    print("Win/Loss/Tie:", str(self.wins_losses()))

    h1 = self.highest_score()
    print("Highest individual score match:", h1["raw"])

    h2 = self.highest_total()
    print("Highest total runs match:", h2["raw"])

    h3 = self.largest_margin()
    print("Largest margin win:", h3["raw"])

    print("LOGS:")
    for x in WorldCupAnalyzer.logs:
        print(x)

# ----- MAIN TEST -----

a = WorldCupAnalyzer("score.txt")
a.report()

```

```

Total matches: 24
Teams: ['India', 'England', 'Pakistan', 'USA', 'Australia', 'Afghanistan',
'SouthAfrica', 'SriLanka', 'NewZealand', 'Bangladesh', 'Netherlands']
Matches played: {'India': 5, 'England': 5, 'Pakistan': 5, 'USA': 3, 'Australia':
5, 'Afghanistan': 3, 'SouthAfrica': 4, 'SriLanka': 4, 'NewZealand': 4,
'Bangladesh': 5, 'Netherlands': 5}
Total runs: {'India': 1465, 'England': 1388, 'Pakistan': 1215, 'USA': 602,
'Australia': 1460, 'Afghanistan': 512, 'SouthAfrica': 1091, 'SriLanka': 808,
'NewZealand': 1099, 'Bangladesh': 1101, 'Netherlands': 1141}
Win/Loss/Tie: {'India': {'wins': 4, 'losses': 1, 'ties': 0}, 'England': {'wins':
4, 'losses': 1, 'ties': 0}, 'Pakistan': {'wins': 2, 'losses': 3, 'ties': 0},
'USA': {'wins': 2, 'losses': 1, 'ties': 0}, 'Australia': {'wins': 4, 'losses':
1, 'ties': 0}, 'Afghanistan': {'wins': 0, 'losses': 3, 'ties': 0},
'SouthAfrica': {'wins': 2, 'losses': 2, 'ties': 0}, 'SriLanka': {'wins': 1,
'losses': 3, 'ties': 0}, 'NewZealand': {'wins': 2, 'losses': 2, 'ties': 0},

```

```
'Bangladesh': {'wins': 0, 'losses': 5, 'ties': 0}, 'Netherlands': {'wins': 3, 'losses': 2, 'ties': 0}}
```

Highest individual score match: India 397 England 280

Highest total runs match: England 340 NewZealand 338

Largest margin win: Australia 367 Afghanistan 123

LOGS:

[1.1] Analyzer ready with 24 matches

```
[6]: %%writefile bollywood.txt
Pathaan|Action|Shah Rukh Khan|Deepika Padukone|250|Sidharth Anand|Released|6.
↳5|1050
Brahmastra|Fantasy|Ranbir Kapoor|Alia Bhatt|300|Ayan Mukerji|Released|6.2|410
Drishyam 2|Thriller|Ajay Devgn|Tabu|90|Abhishek Pathak|Released|8.2|150
KGF Chapter 2|Action|Yash|Srinidhi Shetty|150|Prashanth Neel|Released|8.0|1200
Gangubai Kathiawadi|Drama|Ranveer Singh|Alia Bhatt|80|Sanju Leela
↳Bhansali|Released|7.1|60
Tanhaji|Action|Ajay Devgn|Kajol|120|Om Raut|Released|7.6|320
Satyameva Jayate 2|Action|John Abraham|Divya Khosla|60|Milap Zaveri|Released|3.
↳8|40
Mimi|Comedy/Drama|Nawazuddin Siddiqui|Kriti Sanon|30|Laxman Utekar|Released|7.
↳0|90
Sooryavanshi|Action|Akshay Kumar|Katrina Kaif|160|Rohit Shetty|Released|5.6|300
Bhedhiya|Horror|Varun Dhawan|Kriti Sanon|50|Amar Kaushik|Released|6.6|70
Heropanti 2|Action|Tiger Shroff|Tara Sutaria|80|Ahmed Khan|Released|4.5|35
Shershaah|War|Sidharth Malhotra|Kiara Advani|50|Vishnuvardhan|Released|8.0|200
Rashmi Rocket|Drama|M. Night Shyamalan|Taapsee Pannu|25|Akarsh
↳Khurana|Released|6.8|15
The Kashmir Files|Drama|Anupam Kher|Pallavi Joshi|40|Vivek Agnihotri|Released|7.
↳9|340
Padmaavat|Drama|Shahid Kapoor|Deepika Padukone|215|Sanjay Leela
↳Bhansali|Released|7.0|585
Bajrangi Bhaijaan|Drama|Salman Khan|Kareena Kapoor|90|Kabir Khan|Released|8.
↳1|900
PK|Comedy|Aamir Khan|Anushka Sharma|110|Rajkumar Hirani|Released|8.1|800
Dangal|Sport|Aamir Khan|Sakshi Tanwar|120|Nitesh Tiwari|Released|8.4|2100
Rocky Aur Rani Kii Prem Kahaani|Romance|Ranveer Singh|Alia Bhatt|160|Karan
↳Johar|Released|6.7|140
Upcoming Epic|Drama|New Hero|New Heroine|200|Famous Director|Upcoming|0.0|0
```

Overwriting bollywood.txt

```
[7]: # Single-class Bollywood analyzer (public methods only, no imports, no %
↳formatting)
# Expects bollywood.txt in same folder using '/' separator, ignores lines
↳starting with '#'

class BollywoodAnalyzer:
```

```

# class vars
instances = 0
logs = []

def __init__(self, filename):
    BollywoodAnalyzer.instances += 1
    self.filename = filename
    self.films = []    # list of film dicts
    self.genres = set()
    self.directors = set()
    self.heroes = set()
    self.heroines = set()
    self.load_file()
    self.log("Initialized analyzer with " + str(len(self.films)) + " films")

# simple logger
def log(self, message):
    entry = "[" + str(BollywoodAnalyzer.instances) + "]" + message
    BollywoodAnalyzer.logs.append(entry)

# parse a line into film dict; returns None if invalid or comment
def parse_line(self, line):
    line = line.strip()
    if line == "" or line[0] == "#":
        return None
    parts = line.split("|")
    if len(parts) != 9:
        return None
    title = parts[0].strip()
    genre = parts[1].strip()
    hero = parts[2].strip()
    heroine = parts[3].strip()
    # safe numeric parsing: budget, rating, totalcollection
    try:
        budget = int(parts[4].strip())
    except:
        budget = 0
    director = parts[5].strip()
    status = parts[6].strip()
    try:
        rating = float(parts[7].strip())
    except:
        rating = 0.0
    try:
        collection = int(parts[8].strip())
    except:
        collection = 0

```

```

    film = {
        "title": title,
        "genre": genre,
        "hero": hero,
        "heroine": heroine,
        "budget": budget,
        "director": director,
        "status": status,
        "rating": rating,
        "collection": collection
    }
    return film

# load file (public)
def load_file(self):
    f = open(self.filename, "r")
    for ln in f:
        film = self.parse_line(ln)
        if film is not None:
            self.films.append(film)
            self.genres.add(film["genre"])
            self.directors.add(film["director"])
            self.heroes.add(film["hero"])
            self.heroines.add(film["heroine"])
    f.close()

# ----- Analysis tasks (public methods) -----

def total_films(self):
    return len(self.films)

def list_genres(self):
    return sorted(list(self.genres))

def films_by_director(self, director_name):
    out = []
    for f in self.films:
        if f["director"].lower() == director_name.lower():
            out.append(f)
    return out

def top_grossers(self, top_n):
    films_sorted = sorted(self.films, key=lambda x: x["collection"],
↪reverse=True)
    return films_sorted[:top_n]

def highest_budget_films(self, top_n):

```

```

        films_sorted = sorted(self.films, key=lambda x: x["budget"],
↪reverse=True)
        return films_sorted[:top_n]

    def most_profitable_films(self, top_n):
        # profitability = collection - budget
        films_sorted = sorted(self.films, key=lambda x: (x["collection"] -
↪x["budget"]), reverse=True)
        return films_sorted[:top_n]

    def average_rating(self):
        if len(self.films) == 0:
            return 0.0
        s = 0.0
        count = 0
        for f in self.films:
            if f["rating"] > 0.0:
                s = s + f["rating"]
                count = count + 1
        if count == 0:
            return 0.0
        return s / count

    def films_above_rating(self, threshold):
        out = []
        for f in self.films:
            if f["rating"] >= threshold:
                out.append(f)
        return out

    def underperformers(self):
        # films where collection < budget
        out = []
        for f in self.films:
            if f["collection"] < f["budget"]:
                out.append(f)
        return out

    def top_heroes_by_collection(self, top_n):
        # sum collections per hero
        d = {}
        for f in self.films:
            hero = f["hero"]
            d[hero] = d.get(hero, 0) + f["collection"]
        items = sorted(d.items(), key=lambda x: x[1], reverse=True)
        return items[:top_n]

```

```

def status_counts(self):
    d = {}
    for f in self.films:
        s = f["status"]
        d[s] = d.get(s, 0) + 1
    return d

def director_hits(self):
    # define a 'hit' as rating >= 7.0 and collection >= budget
    d = {}
    for f in self.films:
        if f["rating"] >= 7.0 and f["collection"] >= f["budget"]:
            dir = f["director"]
            d[dir] = d.get(dir, 0) + 1
    return d

# compact report printing many analyses
def report(self):
    print("=== Bollywood Analyzer Report ===")
    print("File: " + self.filename)
    print("Total films: " + str(self.total_films()))
    print("Genres: " + str(self.list_genres()))
    print("Status counts: " + str(self.status_counts()))
    print("Average rating (released films counted): " + str(self.
↪average_rating()))
    print("Top 5 grossers:")
    top = self.top_grossers(5)
    for f in top:
        print(" " + f["title"] + " - collection: " + str(f["collection"]))
    print("Top 5 budgets:")
    tb = self.highest_budget_films(5)
    for f in tb:
        print(" " + f["title"] + " - budget: " + str(f["budget"]))
    print("Top 5 profitable films (collection - budget):")
    prof = self.most_profitable_films(5)
    for f in prof:
        print(" " + f["title"] + " - profit: " + str(f["collection"] -
↪f["budget"]))
    print("Underperformers (collection < budget):")
    up = self.underperformers()
    for f in up:
        print(" " + f["title"] + " - budget: " + str(f["budget"]) + ",
↪collection: " + str(f["collection"]))
    print("Top heroes by total collection:")
    heroes = self.top_heroes_by_collection(5)
    for h, val in heroes:
        print(" " + h + " - total collection: " + str(val))

```

```

print("Director hits count (rating>=7 and collection>=budget):")
dh = self.director_hits()
for k in dh:
    print(" " + k + " - hits: " + str(dh[k]))
print("LOGS:")
for e in BollywoodAnalyzer.logs:
    print(" " + e)
print("=== End of Report ===")

# ----- Run if main -----
if __name__ == "__main__":
    analyzer = BollywoodAnalyzer("bollywood.txt")
    analyzer.report()

```

=== Bollywood Analyzer Report ===

File: bollywood.txt

Total films: 20

Genres: ['Action', 'Comedy', 'Comedy/Drama', 'Drama', 'Fantasy', 'Horror', 'Romance', 'Sport', 'Thriller', 'War']

Status counts: {'Released': 19, 'Upcoming': 1}

Average rating (released films counted): 6.9526315789473685

Top 5 grossers:

Dangal - collection: 2100

KGF Chapter 2 - collection: 1200

Pathaan - collection: 1050

Bajrangi Bhaijaan - collection: 900

PK - collection: 800

Top 5 budgets:

Brahmastra - budget: 300

Pathaan - budget: 250

Padmaavat - budget: 215

Upcoming Epic - budget: 200

Sooryavanshi - budget: 160

Top 5 profitable films (collection - budget):

Dangal - profit: 1980

KGF Chapter 2 - profit: 1050

Bajrangi Bhaijaan - profit: 810

Pathaan - profit: 800

PK - profit: 690

Underperformers (collection < budget):

Gangubai Kathiawadi - budget: 80, collection: 60

Satyameva Jayate 2 - budget: 60, collection: 40

Heropanti 2 - budget: 80, collection: 35

Rashmi Rocket - budget: 25, collection: 15

Rocky Aur Rani Kii Prem Kahaani - budget: 160, collection: 140

Upcoming Epic - budget: 200, collection: 0

Top heroes by total collection:

Aamir Khan - total collection: 2900

Yash - total collection: 1200
 Shah Rukh Khan - total collection: 1050
 Salman Khan - total collection: 900
 Shahid Kapoor - total collection: 585
 Director hits count (rating>=7 and collection>=budget):
 Abhishek Pathak - hits: 1
 Prashanth Neel - hits: 1
 Om Raut - hits: 1
 Laxman Utekar - hits: 1
 Vishnuvardhan - hits: 1
 Vivek Agnihotri - hits: 1
 Sanjay Leela Bhansali - hits: 1
 Kabir Khan - hits: 1
 Rajkumar Hirani - hits: 1
 Nitesh Tiwari - hits: 1
 LOGS:
 [1] Initialized analyzer with 20 films
 === End of Report ===

[8]: *# Extended Bollywood analyzer: single-class, no imports, public methods only.
 # Expects bollywood.txt with '/' separator and same format as provided earlier.*

```
class BollywoodAnalyzer:
    instances = 0
    logs = []

    def __init__(self, filename):
        BollywoodAnalyzer.instances += 1
        self.filename = filename
        self.films = []
        self.genres = set()
        self.directors = set()
        self.heroes = set()
        self.heroines = set()
        self.load_file()
        self.log("Loaded " + str(len(self.films)) + " films")

    def log(self, msg):
        entry = "[" + str(BollywoodAnalyzer.instances) + "] " + msg
        BollywoodAnalyzer.logs.append(entry)

    def parse_line(self, line):
        line = line.strip()
        if line == "" or line[0] == "#":
            return None
        parts = line.split("|")
        if len(parts) != 9:
```

```

        return None
    title = parts[0].strip()
    genre = parts[1].strip()
    hero = parts[2].strip()
    heroine = parts[3].strip()
    try:
        budget = int(parts[4].strip())
    except:
        budget = 0
    director = parts[5].strip()
    status = parts[6].strip()
    try:
        rating = float(parts[7].strip())
    except:
        rating = 0.0
    try:
        collection = int(parts[8].strip())
    except:
        collection = 0
    film = {
        "title": title,
        "genre": genre,
        "hero": hero,
        "heroine": heroine,
        "budget": budget,
        "director": director,
        "status": status,
        "rating": rating,
        "collection": collection
    }
    return film

def load_file(self):
    f = open(self.filename, "r")
    for ln in f:
        film = self.parse_line(ln)
        if film is not None:
            self.films.append(film)
            self.genres.add(film["genre"])
            self.directors.add(film["director"])
            self.heroes.add(film["hero"])
            self.heroines.add(film["heroine"])
    f.close()

# basic tasks
def total_films(self):
    return len(self.films)

```

```

def list_genres(self):
    return sorted(list(self.genres))

def films_by_director(self, director_name):
    out = []
    for f in self.films:
        if f["director"].lower() == director_name.lower():
            out.append(f)
    return out

def top_grossers(self, top_n):
    films_sorted = sorted(self.films, key=lambda x: x["collection"],
↪reverse=True)
    return films_sorted[:top_n]

def highest_budget_films(self, top_n):
    films_sorted = sorted(self.films, key=lambda x: x["budget"],
↪reverse=True)
    return films_sorted[:top_n]

def most_profitable_films(self, top_n):
    films_sorted = sorted(self.films, key=lambda x: (x["collection"] -
↪x["budget"]), reverse=True)
    return films_sorted[:top_n]

def average_rating(self):
    s = 0.0
    count = 0
    for f in self.films:
        if f["rating"] > 0.0:
            s = s + f["rating"]
            count = count + 1
    if count == 0:
        return 0.0
    return s / count

def films_above_rating(self, threshold):
    out = []
    for f in self.films:
        if f["rating"] >= threshold:
            out.append(f)
    return out

def underperformers(self):
    out = []
    for f in self.films:

```

```

        if f["collection"] < f["budget"]:
            out.append(f)
    return out

def top_heroes_by_collection(self, top_n):
    d = {}
    for f in self.films:
        hero = f["hero"]
        d[hero] = d.get(hero, 0) + f["collection"]
    items = sorted(d.items(), key=lambda x: x[1], reverse=True)
    return items[:top_n]

def status_counts(self):
    d = {}
    for f in self.films:
        s = f["status"]
        d[s] = d.get(s, 0) + 1
    return d

def director_hits(self):
    d = {}
    for f in self.films:
        if f["rating"] >= 7.0 and f["collection"] >= f["budget"]:
            dir = f["director"]
            d[dir] = d.get(dir, 0) + 1
    return d

# ----- NEW TASKS -----

# 1. ROI percent per film (returns list of (title, roi_percent))
def roi_list(self):
    out = []
    for f in self.films:
        if f["budget"] == 0:
            roi = None
        else:
            roi = (f["collection"] - f["budget"]) * 100.0 / f["budget"]
        out.append((f["title"], roi))
    return out

# 2. Top N films by ROI (ignore films with budget 0)
def top_by_roi(self, top_n):
    vals = []
    for f in self.films:
        if f["budget"] > 0:
            roi = (f["collection"] - f["budget"]) * 100.0 / f["budget"]
            vals.append((f, roi))

```

```

        vals_sorted = sorted(vals, key=lambda x: x[1], reverse=True)
        return vals_sorted[:top_n]

# 3. Profit margin absolute and percent for each film
def profit_info(self):
    out = []
    for f in self.films:
        profit = f["collection"] - f["budget"]
        if f["budget"] == 0:
            percent = None
        else:
            percent = profit * 100.0 / f["budget"]
        out.append({"title": f["title"], "profit": profit, "percent":
↪percent})
    return out

# 4. Count profitable (collection >= budget)
def count_profitable(self):
    c = 0
    for f in self.films:
        if f["collection"] >= f["budget"] and f["budget"] > 0:
            c = c + 1
    return c

# 5. Average budget per genre
def avg_budget_by_genre(self):
    sums = {}
    counts = {}
    for f in self.films:
        g = f["genre"]
        sums[g] = sums.get(g, 0) + f["budget"]
        counts[g] = counts.get(g, 0) + 1
    out = {}
    for g in sums:
        out[g] = float(sums[g]) / counts[g]
    return out

# 6. Average collection per genre
def avg_collection_by_genre(self):
    sums = {}
    counts = {}
    for f in self.films:
        g = f["genre"]
        sums[g] = sums.get(g, 0) + f["collection"]
        counts[g] = counts.get(g, 0) + 1
    out = {}
    for g in sums:

```

```

        out[g] = float(sums[g]) / counts[g]
    return out

# 7. Director success rate = profitable films / total films for that
↪director
def director_success_rate(self):
    totals = {}
    profitable = {}
    for f in self.films:
        d = f["director"]
        totals[d] = totals.get(d, 0) + 1
        if f["collection"] >= f["budget"] and f["budget"] > 0:
            profitable[d] = profitable.get(d, 0) + 1
    out = {}
    for d in totals:
        p = profitable.get(d, 0)
        out[d] = float(p) / totals[d]
    return out

# 8. Hero / heroine average rating and average collection
def actor_heroine_stats(self):
    hero_stats = {}
    heroine_stats = {}
    for f in self.films:
        h = f["hero"]
        hero_stats[h] = hero_stats.get(h, {"rating_sum":0.0, "rating_count":
↪0, "collection":0})
        if f["rating"] > 0.0:
            hero_stats[h]["rating_sum"] = hero_stats[h]["rating_sum"] +
↪f["rating"]
            hero_stats[h]["rating_count"] = hero_stats[h]["rating_count"] +
↪1
            hero_stats[h]["collection"] = hero_stats[h]["collection"] +
↪f["collection"]

        hr = f["heroine"]
        heroine_stats[hr] = heroine_stats.get(hr, {"rating_sum":0.0,
↪"rating_count":0, "collection":0})
        if f["rating"] > 0.0:
            heroine_stats[hr]["rating_sum"] =
↪heroine_stats[hr]["rating_sum"] + f["rating"]
            heroine_stats[hr]["rating_count"] =
↪heroine_stats[hr]["rating_count"] + 1
            heroine_stats[hr]["collection"] = heroine_stats[hr]["collection"] +
↪f["collection"]

```

```

        # compute averages
        hero_out = {}
        for k in hero_stats:
            rcount = hero_stats[k]["rating_count"]
            if rcount == 0:
                avg_r = 0.0
            else:
                avg_r = hero_stats[k]["rating_sum"] / rcount
            hero_out[k] = {"avg_rating": avg_r, "total_collection":
↪hero_stats[k]["collection"]}

        heroine_out = {}
        for k in heroine_stats:
            rcount = heroine_stats[k]["rating_count"]
            if rcount == 0:
                avg_r = 0.0
            else:
                avg_r = heroine_stats[k]["rating_sum"] / rcount
            heroine_out[k] = {"avg_rating": avg_r, "total_collection":
↪heroine_stats[k]["collection"]}

        return {"heroes": hero_out, "heroines": heroine_out}

# 9. Rating distribution buckets
def rating_buckets(self):
    buckets = {"0-3":0, "3-5":0, "5-7":0, "7-9":0, "9-10":0}
    for f in self.films:
        r = f["rating"]
        if r < 3.0:
            buckets["0-3"] = buckets["0-3"] + 1
        elif r < 5.0:
            buckets["3-5"] = buckets["3-5"] + 1
        elif r < 7.0:
            buckets["5-7"] = buckets["5-7"] + 1
        elif r < 9.0:
            buckets["7-9"] = buckets["7-9"] + 1
        else:
            buckets["9-10"] = buckets["9-10"] + 1
    return buckets

# 10. Top rated films that underperformed (rating >= threshold but
↪collection < budget)
def top_rated_underperformers(self, rating_threshold):
    out = []
    for f in self.films:
        if f["rating"] >= rating_threshold and f["collection"] <
↪f["budget"]:
```

```

        out.append(f)
    out_sorted = sorted(out, key=lambda x: x["rating"], reverse=True)
    return out_sorted

# 11. Films with highest and lowest ROI (ignore budget 0)
def highest_lowest_roi(self):
    vals = []
    for f in self.films:
        if f["budget"] > 0:
            roi = (f["collection"] - f["budget"]) * 100.0 / f["budget"]
            vals.append((f, roi))
    if not vals:
        return None, None
    vals_sorted = sorted(vals, key=lambda x: x[1], reverse=True)
    return vals_sorted[0], vals_sorted[-1]

# 12. Upcoming or non-released films and their total planned budget
def upcoming_films_and_budget(self):
    out = []
    total_budget = 0
    for f in self.films:
        if f["status"].lower() != "released":
            out.append(f)
            total_budget = total_budget + f["budget"]
    return out, total_budget

# ----- Compact combined report -----
def extended_report(self):
    print("=== Extended Bollywood Report ===")
    print("Total films: " + str(self.total_films()))
    print("Status counts: " + str(self.status_counts()))
    print("Average rating: " + str(self.average_rating()))
    print("")
    print("Top 5 grossers:")
    for f in self.top_grossers(5):
        print(" " + f["title"] + " - collection " + str(f["collection"]))
    print("")
    print("Top 5 by ROI:")
    top_roi = self.top_by_roi(5)
    for item, roi in top_roi:
        print(" " + item["title"] + " - ROI percent " + str(roi))
    print("")
    print("Average budget by genre:")
    abg = self.avg_budget_by_genre()
    for g in abg:
        print(" " + g + " : " + str(abg[g]))
    print("")

```

```

print("Director success rates (profitable fraction):")
ds = self.director_success_rate()
for d in ds:
    print(" " + d + " : " + str(ds[d]))
print("")
print("Rating buckets: " + str(self.rating_buckets()))
print("")
print("Top rated underperformers (rating >= 7):")
tr = self.top_rated_underperformers(7.0)
for f in tr:
    print(" " + f["title"] + " rating " + str(f["rating"]) + "␣
↳collection " + str(f["collection"]) + " budget " + str(f["budget"]))
    print("")
    upcoming, total_up_budget = self.upcoming_films_and_budget()
    print("Upcoming films count: " + str(len(upcoming)) + " total planned␣
↳budget: " + str(total_up_budget))
    print("=== End Extended Report ===")

# ----- Run example -----
if __name__ == "__main__":
    analyzer = BollywoodAnalyzer("bollywood.txt")
    analyzer.extended_report()
    print("LOGS:")
    for e in BollywoodAnalyzer.logs:
        print(" " + e)

```

=== Extended Bollywood Report ===

Total films: 20

Status counts: {'Released': 19, 'Upcoming': 1}

Average rating: 6.9526315789473685

Top 5 grossers:

Dangal - collection 2100
 KGF Chapter 2 - collection 1200
 Pathaan - collection 1050
 Bajrangi Bhaijaan - collection 900
 PK - collection 800

Top 5 by ROI:

Dangal - ROI percent 1650.0
 Bajrangi Bhaijaan - ROI percent 900.0
 The Kashmir Files - ROI percent 750.0
 KGF Chapter 2 - ROI percent 700.0
 PK - ROI percent 627.2727272727273

Average budget by genre:

Action : 136.66666666666666

Fantasy : 300.0
Thriller : 90.0
Drama : 108.33333333333333
Comedy/Drama : 30.0
Horror : 50.0
War : 50.0
Comedy : 110.0
Sport : 120.0
Romance : 160.0

Director success rates (profitable fraction):

Sidharth Anand : 1.0
Ayan Mukerji : 1.0
Abhishek Pathak : 1.0
Prashanth Neel : 1.0
Sanju Leela Bhansali : 0.0
Om Raut : 1.0
Milap Zaveri : 0.0
Laxman Utekar : 1.0
Rohit Shetty : 1.0
Amar Kaushik : 1.0
Ahmed Khan : 0.0
Vishnuvardhan : 1.0
Akarsh Khurana : 0.0
Vivek Agnihotri : 1.0
Sanjay Leela Bhansali : 1.0
Kabir Khan : 1.0
Rajkumar Hirani : 1.0
Nitesh Tiwari : 1.0
Karan Johar : 0.0
Famous Director : 0.0

Rating buckets: {'0-3': 1, '3-5': 2, '5-7': 6, '7-9': 11, '9-10': 0}

Top rated underperformers (rating >= 7):

Gangubai Kathiawadi rating 7.1 collection 60 budget 80

Upcoming films count: 1 total planned budget: 200

=== End Extended Report ===

LOGS:

[1] Loaded 20 films

[]: